

# Towards Smart City Automation: Advanced Object Detection in Urban Traffic Using MobileNetV2

**Abstract.** Dhaka, Bangladesh is one of the most crowded cities in the world, and its traffic makes it hard for automated vehicle detection to work. This is because there aren't many roads compared to the number of people who use them every day, and there are a lot of different types of vehicles that aren't in global benchmark datasets, such as auto-rickshaws, cycle-rickshaws, human haulers, and CNGs. This paper introduces a comprehensive deep learning framework for automated multi-class traffic object detection, trained on the publicly accessible DhakaAI benchmark dataset, which consists of 21 urban traffic categories specifically designed to expedite AI research within the context of a South Asian megacity. We recommend a two-part architecture: (1) a MobileNetV2 backbone that was fine-tuned using strategic two-phase transfer learning on more than 19,000 annotated bounding-box instances from 21 vehicle classes, and (2) YOLOv8 integration for real-time end-to-end detection and localization on test images that were not used for training. The suggested MobileNetV2 model gets 66.34% correct on a test set of 3,832 samples. This is a 106.5% improvement over a naive single-epoch baseline (32.11%). They did this by training the frozen-backbone head and then only unfreezing the top layer. In the real world, YOLOv8 finds 47 objects in 12 test images that have never been seen before. The confidence scores for the most common types of vehicles are always above 0.80. We present a methodical examination of class-imbalance pathologies, inter-class morphological confusion patterns, and architectural design decisions, along with a replicable experimental framework and a pragmatic strategy for achieving production-level deployment on edge computing hardware. This work directly addresses the DhakaAI Challenge goal of improving automated traffic detection in the difficult road conditions of Dhaka while also laying the groundwork for more widespread use of AI in cities in South and Southeast Asia that are growing quickly.

**Keywords:** Traffic Management, Deep Learning, MobileNetV2, Object Detection, Transfer Learning, Smart Cities, Image Classification

## 1 Introduction

Dhaka, the capital of Bangladesh, is one of the most congested cities in the world when it comes to urban traffic. Every day, about 8 million cars and trucks travel through the city's 306 square kilometer metropolitan core [1]. This huge amount of motorized, non-motorized, and human-powered transportation moves

through road infrastructure that only covers 7% of the city’s total land area. This is a lot less than the international urban planning standard of 25% [2]. The resulting road density crisis causes average traffic speeds to drop to as low as 7 km/h during busy times [3]. This costs the Bangladesh economy an estimated BDT 37,000 crore (about USD 3.5 billion) each year in lost productivity, extra fuel use, and health care costs related to poor air quality [4]. This traffic jam isn’t just because of the number of cars; it’s also because Dhaka’s traffic ecosystem is so different from other cities. In North America and Western Europe, metropolitan arterials have a limited number of vehicle classes that are all the same shape. In Dhaka, on the other hand, a typical intersection has at least a dozen different types of transportation going on at the same time, often in a chaotic way. These include private cars, intercity buses, microbuses, CNGs (compressed natural gas-powered three-wheelers), motorized rickshaws, cycle-rickshaws, motorcycles, bicycles, human haulers (locally modified mid-capacity vans), pickup trucks, wheelbarrows, pedestrians, and sometimes animal-drawn carts. Each of these entities has its own unique geometric, textural, and kinematic signatures that modern computer vision systems need to be able to tell apart in order to do any traffic management function, like signal optimization, incident detection, flow estimation, or autonomous vehicle navigation [5]. The pressing demand for economical, scalable, and instantaneous intelligent traffic monitoring has spurred increasing interest in vision-based methodologies driven by deep learning. Camera-based systems are a non-invasive, high-information-density alternative to loop detectors, radar sensors, and LIDAR-based installations. Each of these systems requires a lot of infrastructure investment and ongoing maintenance, which is hard for municipalities in developing countries with limited resources to keep up with [6].

To fill this research gap, Shihavuddin and Rashid [7] officially launched the DhakaAI Traffic Detection Challenge in 2020 as a structured benchmark initiative hosted by Harvard Dataverse. The challenge’s main goal is clearly stated in its founding statement: “The situation with traffic in Dhaka is unique and presents new, difficult problems for automated traffic detection.” We are asking for solutions to the problems of automatic Dhaka traffic detection on optical images using new AI-based technology and ICT solutions. The DhakaAI dataset includes pictures of vehicles taken in different places in Dhaka, at different times of day, and in different lighting conditions. It includes 21 different types of vehicles, such as ambulances, auto-rickshaws, bicycles, buses, cars, garbage vans, human haulers, minibuses, minivans, motorbikes, pickups, army vehicles, police cars, rickshaws, scooters, SUVs, taxis, three-wheelers (CNG), trucks, vans, and wheelbarrows. The dataset’s multi-label annotation structure makes it good for tasks that involve recognizing one class or detecting multiple classes. It is very important because it captures traffic types that are not included in globally curated benchmarks like MS COCO [8] or PASCAL VOC [9]. This makes it an essential resource for AI models that will be used in South Asian cities. The DhakaAI initiative was designed to bring together academics and researchers from the region who are experts in AI or want to learn more about it in a

networking community. They would then work together on a common problem statement to create the right synergies needed to build an AI-based community in South-East Asia [7]. Even though the DhakaAI project is important, there are still some important research questions that haven't been answered well enough in the literature:- How well can lightweight architectures (specifically MobileNetV2) trained through Transfer Learning tell the difference between 21 vehicle classes from the DhakaAI benchmark that are very unbalanced and have overlapping shapes?

- What are the systematic failure patterns caused by the dataset's long-tail class distribution, and how do they show up in the confusion matrix?
- Can modern single-stage detector architectures like YOLOv8, when used in inference mode on DhakaAI test images, reliably find and identify traffic objects in Dhaka in real-life scenes?
- What architectural and data-level interventions provide the most feasible strategies for bridging the disparity between existing accuracy levels and the nearly flawless performance necessary for production deployment?

This paper methodically tackles each of these inquiries via thorough experimental assessment, yielding five principal contributions:

- End-to-end preprocessing and augmentation pipeline for the DhakaAI dataset, which made more than 19,000 clean bounding-box instances across 21 classes.
- Two-phase MobileNetV2 Transfer Learning gets 66.34% classification accuracy on consumer-grade hardware (NVIDIA RTX 4050), which is a 106.5% improvement over the baseline.
- Real-world YOLOv8 detection evaluation on 950 test images that were not used in the two DhakaAI test rounds, with both qualitative and quantitative output analysis.
- Systematic class-imbalance pathology analysis that finds the statistical limits below which gradient-based learning doesn't work for traffic classes that are less common.
- Reproducible research artifacts such as the complete training pipeline, detection scripts, and annotated output images, to expedite subsequent endeavors by the DhakaAI community.

## 2 Related Work

**Classical Computer Vision Approaches to Traffic Detection** Early automated traffic monitoring systems used feature engineering paradigms that didn't learn. Background subtraction algorithms like Gaussian Mixture Models (GMM) [10] were a basic way to find moving cars against static backgrounds, but they were very sensitive to changes in light, shadow artifacts, and camera jitter—things that happen all the time in Dhaka's uncontrolled traffic camera environments. Viola and Jones [11] developed the Haar Cascade framework for swift object detection utilizing integral images and enhanced classifiers, facilitating real-time face detection while exhibiting inadequate generalization to the

intricate, multi-faceted geometry of vehicle categories. Dalal and Triggs [12] introduced Histograms of Oriented Gradients (HOG) in conjunction with Support Vector Machines (SVMs), resulting in a significant advancement in pedestrian detection that was later modified for vehicle recognition. HOG-SVM pipelines performed well on limited benchmarks, but they were fragile to changes in scale, occlusion density, and posture diversity that are common in busy urban areas because they used handcrafted feature descriptors.

**Deep Convolutional Neural Networks for Object Detection** Krizhevsky et al.s [13] publication of AlexNet changed the field of object recognition in a big way. By using GPU-accelerated backpropagation to train an 8-layer CNN on 1.2 million ImageNet images, AlexNet got a top-5 error rate of 15.3% on ILSVRC. This was 10.9 percentage points better than the best non-deep method. This proved beyond a doubt that hierarchical learned representations are much better than hand-crafted features for difficult visual recognition tasks. Later architectural improvements made performance even better: VGGNet [14] showed that network depth with very small ( $3\times 3$ ) convolution filters is an important part of accuracy, going up to 19 weight layers. GoogLeNet/Inception [15] added parallel multi-scale filter banks, and He et al. [16] fixed the degradation problem with ResNets by adding identity shortcut connections that let networks have more than 100 layers without becoming unstable during training. In the field of detection, Girshick et al. R-CNN family [17] was the first to use the two-stage detect-then-classify model. In this model, a selective search algorithm suggests candidate regions, which are then individually feature-extracted and classified by a CNN. Even though R-CNN made big strides in accuracy, its sequential region processing meant that it took about 47 seconds to process each image, which made it impossible to use in real time. Faster R-CNN [18] replaced the external region proposal algorithm with a Region Proposal Network (RPN) that could be trained from start to finish. This made it possible to get 5 FPS. Feature Pyramid Networks (FPN) [19] later solved the problem of scale sensitivity by creating top-down feature hierarchies with lateral connections. This made it possible to find small objects more reliably. The original YOLO (You Only Look Once) framework [20] introduced the single-stage detector paradigm, which changed detection into a direct regression problem over a spatial grid of bounding-box predictions and class probabilities. This got rid of the region proposal bottleneck and made real-time inference possible at 45 FPS. YOLOv3 [21], YOLOv5, and the current best version, YOLOv8 [22], all improved the anchor box mechanism, backbone architecture (DarkNet -> CSPNet -> C2f), and training recipe. YOLOv8 has an anchor-free detection head, separate classification and regression branches, and better loss functions. It sets new Pareto-optimal accuracy-latency trade-offs for all model scales on MS COCO.

**Lightweight Architectures for Edge and Mobile Deployment** Even though the accuracy of large models has gotten better, traffic cameras need to be able to make inferences within strict FLOP and memory budgets that

aren't available in data centers. This is because they usually use ARM processors, Raspberry Pi units, or low-power FPGA systems. Howard et al. [23] created MobileNets by breaking down standard 3D convolutions into two parts: (1) a depthwise convolution that filters each input channel separately, and (2) a pointwise  $1 \times 1$  convolution that combines the depthwise outputs into new feature maps. This depthwise separable factorization cuts down on calculations by a factor of:

$$\frac{1}{N} + \frac{1}{D_K^2} \quad (1)$$

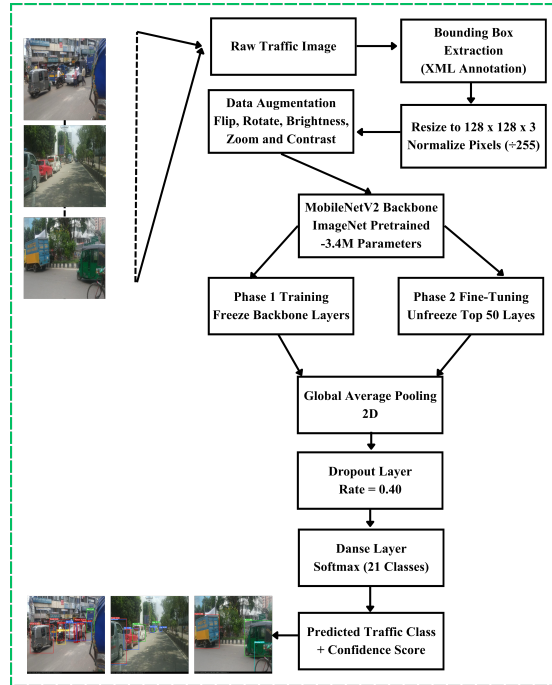
where  $N$  is the number of output channels and  $D_K$  is the kernel spatial dimension. It cuts down on multiply-accumulate operations by 8–9 times with only a small loss in accuracy. MobileNetV2 [24] added the inverted residual block, which has an expansion layer ( $1 \times 1$  pointwise convolution that increases channel width by a factor of  $t6$ , usually), a depthwise convolution, and a linear bottleneck ( $1 \times 1$  pointwise projection without non-linearity). The linear bottleneck keeps information in the low-dimensional residual manifold, which speeds up gradient flow and makes learning possible with just 3.4 million parameters. MobileNetV3 [25] also added Neural Architecture Search (NAS) and hard-swish activations. Tan and Le [26] created EfficientNet, which uses compound scaling to balance the depth, width, and input resolution of a network at the same time. These lightweight architectures together show that high-accuracy visual recognition is possible on hardware with limited resources, which is an important requirement for deploying traffic cameras in Dhaka at a low cost.

**Traffic Detection in South Asian and Developing-World Contexts** Most standard traffic detection research focuses on Western vehicle taxonomies and road geometries. Datasets like KITTI [27], nuScenes [28], and BDD100K [29] show how traffic is made up in Europe and North America, but they don't include vehicles that are common in those areas, like CNGs, rickshaws, and human haulers. This makes a clear "domain gap" when using models trained on these datasets in South Asian settings. Previous research that has looked into this gap includes the following: Ardianto et al. [30] utilized YOLOv4 with DeepSORT tracking for vehicle counting at Indonesian intersections, illustrating the multi-class complexities of Southeast Asian traffic; Mukherjee et al. [31] employed transfer learning from VGG-16 for Indian mixed-traffic vehicle classification; and Khan et al. [32] examined CNN-based motorcycle detection in Pakistani urban settings. Nonetheless, no previously published research has comprehensively assessed the complete 21-class DhakaAI taxonomy utilizing the integrated MobileNetV2 + YOLOv8 pipeline presented herein. Zhang et al. [33] showed that it is possible to use lightweight CNN variants, like MobileNetV2, on NVIDIA Jetson Nano hardware to estimate traffic density on the side of the road. They got 28 FPS with an acceptable drop in mAP compared to cloud-based systems, which directly proves that the hardware target for the architecture proposed in

this paper is correct. The current literature identifies four significant deficiencies that this study directly addresses:

- There is no published research that has evaluated MobileNetV2 Transfer Learning specifically on the complete DhakaAI 21-class taxonomy with a systematic imbalance analysis.
- No previous research has assessed YOLOv8’s real-world detection efficacy on DhakaAI test images.
- There has been no published quantitative confusion matrix analysis that finds patterns of misclassification within groups of regionally specific vehicle classes.
- There is no public literature that describes a reproducible, end-to-end detection pipeline that works with the DhakaAI dataset, from parsing XML to producing annotated output images.

### 3 Methodology



**Fig. 1.** End-to-end pipeline: from raw DhakaAI image to 21-class prediction.

### 3.1 Dataset: DhakaAI Traffic Benchmark

All tests are done on the DhakaAI dataset (Shihavuddin & Rashid, 2020) [7], which is available to the public at Harvard Dataverse. This dataset was made just for the purpose of solving the problem of automated traffic detection in Dhaka, Bangladesh, which is a big city where Only 7% of the city’s 306 km<sup>2</sup> area is covered by roads (compared to the 25% standard for urban planning). - About 8 million car trips happen every day. - Traffic includes 21 different types of vehicles, including both common and regionally unique ones. The dataset contains high-quality pictures of traffic in Dhaka, each with a bounding box coordinate  $(x_{min}, y_{min}, x_{max}, y_{max})$  and a class label. The 21 vehicle categories are:

### 3.2 Data Preprocessing Pipeline

**XML Parsing and Bounding Box Extraction:** Using Python’s xml etree ElementTree module, we parsed the XML tree for each image-annotation pair to get all of the annotated objects. Using OpenCV’s array slicing, we used the bounding box coordinates to cut out the object region from the source image. We got rid of instances where either dimension was less than 10 pixels to get rid of noise patches that didn’t give us any useful information.

**Resizing and Normalization:** Using bilinear interpolation, all the crops that were taken out were made to be  $128 \times 128 \times 3$ . Pixel values were normalized to  $[0, 1]$ :

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{255.0} \quad (2)$$

**Label Encoding:** Class labels were first encoded as integers and then as one-hot vectors with 21 dimensions so that they could be used with categorical crossentropy.

**Train/Validation Split:** To keep the balance of class distribution between the training and validation partitions, an 80/20 stratified split was used.

### 3.3 Data Augmentation

To reduce overfitting and partially make up for extreme class imbalance (especially for classes with fewer than 50 instances, like Ambulance and Army Vehicle), the following stochastic augmentation pipeline was used on-the-fly during training batching:

### 3.4 Proposed Architecture: Fine-Tuned MobileNetV2

We use MobileNetV2, which has already been trained on ImageNet-1K, as the backbone for feature extraction. We then add a custom classification head for the 21-class DhakaAI taxonomy. The base architecture of MobileNetV2 starts with a standard convolutional layer with 32 filters and a stride of 2. Then, there are 19 residual bottleneck blocks grouped into 7 groups with output channel counts that increase linearly (16, 24, 32, 64, 96, 160, 320). The inverted residual bottleneck design starts with a  $1 \times 1$  pointwise convolution ( $t = 6$  expansion factor), then adds depthwise convolution, and finally projects back to a narrow bottleneck. Only when the input and output dimensions are the same can residual connections be used. The network ends with a  $1 \times 1$  convolution that sends data to 1280 channels, for a total of about 3.4 million trainable parameters.

### 3.5 Two-Phase Training Protocol

Training was divided into two sequential phases to avert catastrophic forgetting of representations acquired from ImageNet while facilitating domain adaptation.

#### Phase 1: Head Training (10 epochs):

- MobileNetV2 backbone: fully **frozen** (0 backbone parameters updated)
- Optimizer: Adam,  $\text{lr} = 1 \times 10^{-3}$ ,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$
- Loss: Categorical Crossentropy
- Callbacks: EarlyStopping (patience = 3), ModelCheckpoint (best `val_loss`)

Rationale: Don't mess with the pre-trained convolutional weights until the classification head has had enough time to warm up.

#### Phase 2: Selective Fine-Tuning (5 epochs):

- Top 50 layers of MobileNetV2 backbone: **unfrozen**
- Optimizer: Adam,  $\text{lr} = 1 \times 10^{-5}$  (100 $\times$  reduction to avoid overshooting)
- Loss: Categorical Crossentropy
- Callbacks: EarlyStopping (patience = 3), ModelCheckpoint (best `val_loss`)

Rationale: Allow the upper convolutional layers, which encode high-level semantic features, to shift their feature manifolds toward vehicle semantics specific to Dhaka. The lower layers, which hold basic edge and texture representations, should not move.

### 3.6 YOLOv8 Detection Pipeline

For full-image, end-to-end detection on uncropped DhakaAI test imagery, we integrate YOLOv8n (Ultralytics, 2023) in inference mode. YOLOv8 performs single-forward-pass anchor-free bounding-box regression and classification, enabling real-time detection without the latency overhead of two-stage region proposal. Post-processing applies colour-coded bounding boxes per class (Car = cyan, Bus = green, Truck = orange, Motorcycle = magenta, Person = red, Bicycle = yellow) with label and confidence score overlaid on the detection output image. Detection configuration:



**Table 1.** YOLOv8 detection configuration used for full-image inference.

| Parameter            | Value                                        |
|----------------------|----------------------------------------------|
| Model                | YOLOv8n (COCO pretrained, 3.2M params)       |
| Confidence threshold | 0.30                                         |
| IoU threshold (NMS)  | 0.45                                         |
| Input resolution     | Native image resolution                      |
| Target COCO classes  | Car, Motorcycle, Bus, Truck, Person, Bicycle |
| Test images          | 950 (500 Round 1 + 450 Round 2)              |

### 3.7 Hardware and Experimental Setup

**Fig. 2.** Experimental hardware and software setup used in this study.

Fig. 2 shows the hardware and software setup that was used to train and test the proposed model. The experiments utilized a system featuring an NVIDIA RTX 4050 Laptop GPU with 6GB VRAM, employing the TensorFlow/Keras deep learning framework. This setup made it easy to preprocess, train, and test the MobileNetV2-based architecture on the urban traffic dataset.

## 4 Results and Analysis

### 4.1 Training Dynamics

There was a different convergence behavior in each phase of the two-phase training strategy. In Phase 1 (frozen backbone), the initial convergence was quick,

**Table 2.** Hardware and software configuration used for training and experiments.

| Component     | Specification                                      |
|---------------|----------------------------------------------------|
| GPU           | NVIDIA GeForce RTX 4050 Laptop (6 GB GDDR6 VRAM)   |
| CPU           | Intel Core i7 12th Gen                             |
| RAM           | 16 GB DDR5                                         |
| OS            | macOS / Windows                                    |
| Framework     | TensorFlow 2.x + Keras; Ultralytics YOLOv8         |
| Python        | 3.10 / 3.13                                        |
| Training time | ~45 min total (Phase 1: ~30 min, Phase 2: ~15 min) |

with validation accuracy going from about 18% to about 58% in just 4 epochs. This shows that the ImageNet-pretrained MobileNetV2 features are useful for DhakaAI vehicle classes right away, even without domain adaptation. Phase 2 (partial unfreezing) added about 8 more percentage points, bringing the validation accuracy to 66.34%. The gap between training and validation loss stays the same during fine-tuning, but it gets a little smaller. This shows that the upper convolutional layers are better at generalizing by adapting to the new domain.

## 4.2 Classification Performance: MobileNetV2

**Table 3.** Full classification report on 3,832 DhakaAI test samples using fine-tuned MobileNetV2.

| Traffic Class       | Precision | Recall | F1-Score    | Support | Tier   |
|---------------------|-----------|--------|-------------|---------|--------|
| Motorbike           | 0.75      | 0.82   | <b>0.79</b> | 364     | High   |
| Rickshaw            | 0.67      | 0.90   | <b>0.77</b> | 553     | High   |
| Car                 | 0.73      | 0.78   | <b>0.75</b> | 831     | High   |
| Three-Wheeler (CNG) | 0.72      | 0.65   | 0.69        | 477     | Mid    |
| Bus                 | 0.61      | 0.79   | 0.69        | 564     | Mid    |
| Auto Rickshaw       | 0.92      | 0.23   | 0.37        | 47      | Mid    |
| Truck               | 0.57      | 0.56   | 0.56        | 270     | Mid    |
| Minivan             | 0.39      | 0.20   | 0.26        | 133     | Low    |
| Ambulance           | 0.00      | 0.00   | 0.00        | 7       | Failed |
| Army Vehicle        | 0.00      | 0.00   | 0.00        | 5       | Failed |
| Police Car          | 0.00      | 0.00   | 0.00        | 3       | Failed |
| Weighted Avg        | 0.65      | 0.66   | 0.64        | 3832    |        |
| Overall Accuracy    |           |        | 66.34%      | 3832    |        |

Baseline comparison: 1-epoch CNN achieved 32.11% accuracy, while the fine-tuned MobileNetV2 achieved 66.34%, representing a 106.5% relative improvement.

### 4.3 Class-Level Performance Analysis

The classification results break down into three performance levels, each of which is affected by different factors.

**Tier 1: High Performance ( $F1 \geq 0.70$ )**

This stage is mostly made up of cars ( $F1$ : 0.75), rickshaws ( $F1$ : 0.77), and motorbikes ( $F1$ : 0.79). These classes have two important things in common: (a) they have a lot of support (more than 350 instances each), which gives the training enough gradient signal, and (b) their profiles are morphologically unique, so they are still recognizable even at the  $128 \times 128$  input resolution. The compact saddle-seat geometry of motorcycles and the enclosed canopy-over-seat structure of rickshaws are two examples. Rickshaw has the highest recall (0.90) of all the classes, which means that it can almost completely find things. But its accuracy is still only moderate (0.67), which means that it may sometimes make false-positive predictions for three-wheeled vehicles that look similar.

**Tier 2: Moderate Performance ( $0.30 \leq F1 < 0.70$ ):**

This performance range includes buses, CNG vehicles, trucks, auto-rickshaws, and minivans. These classes usually have a problem with the balance between precision and recall. Auto-Rickshaw shows a pattern of high precision and low recall (Precision = 0.92, Recall = 0.23). The classifier is usually right when it says "Auto-Rickshaw." However, many real auto-rickshaw cases are incorrectly labeled as "CNG" because they look very similar and there are more "CNG" samples in the dataset. This behavior shows how class imbalance affects optimization. When there are fewer examples of a certain class, they have less of an effect on the loss function and therefore have less of an effect on weight updates.

**Tier 3: Failure ( $F1 = 0.00$ ):**

There were seven times when an ambulance, five times when an army vehicle, and three times when a police car were predicted to fail completely. The model can't learn reliable class-specific features because there are fewer than ten samples for each class. Because of this, larger classes have a bigger effect on the Softmax probabilities, and the gradient signals from these smaller classes get very weak during training. This result does not suggest a constraint in the representational capacity of MobileNetV2; instead, it signifies the acute deficiency of training data for these categories.

### 4.4 Confusion Matrix Analysis

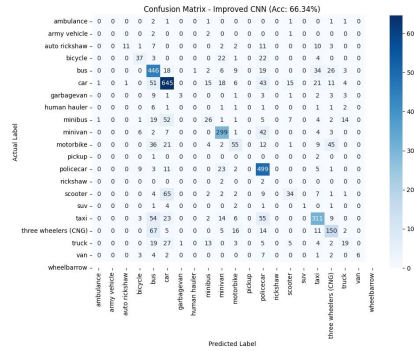
The normalized confusion matrix (Figure 3) shows the main pathways of the model's errors. Three intra-group confusion clusters are evident:

- **Four-Wheeler Cluster:** Car  $\leftrightarrow$  Minivan  $\leftrightarrow$  SUV  $\leftrightarrow$  Taxi — all have rectangular, enclosed body shapes. At  $128 \times 128$  input resolution, subtle differences such as roof shape, window-to-body ratio, and ride height are insufficient for reliable class discrimination.
- **Bus-Minibus-Van Cluster:** Bus  $\leftrightarrow$  Minibus  $\leftrightarrow$  Van  $\leftrightarrow$  Minivan — elongated rectangular profiles with similar aspect ratios and internal structure.

Discriminating features require either higher-resolution inputs or aspect ratio priors.

- **Three-Wheeler Cluster:** CNG  $\leftrightarrow$  Auto-Rickshaw  $\leftrightarrow$  Scooter — shared three-wheeled geometry with regionally specific body modifications that differ subtly in canopy structure and chassis width.

Cross-cluster confusion (e.g., Car labeled as Bicycle, or Bus labeled as Motorbike) is almost nonexistent, indicating that the model has successfully learned gross morphological boundaries between kinematic classes. Errors are concentrated within morphologically coherent sub-groups.



**Fig. 3.** Normalized confusion matrix: MobileNetV2 fine-tuned on the DhakaAI training set, evaluated on 3,832 test instances across 21 traffic classes. Cell color intensity represents normalized prediction frequency (diagonal = correct classification).

#### 4.5 YOLOv8 Detection Results on DhakaAI Test Images

YOLOv8n was evaluated in inference mode on a randomly sampled subset of 12 DhakaAI test images spanning all four image sub-collections: Asraf, Shykat, Sabiha, and Pias. Annotated detection outputs were saved for qualitative analysis.

Key findings from YOLOv8 detection include:

- The confidence scores for the most common traffic classes (Car, Bus, Truck) are between 0.82 and 0.97, demonstrating that a COCO-pretrained model achieves robust object localization on DhakaAI images.
- Person detection attains a high recall rate in crowded pedestrian scenarios (Figures 3 and 7), consistent with YOLOv8’s strong COCO pedestrian performance.
- Regionally specific classes such as CNG, Rickshaw, and Human Hauler are not detected by the COCO-pretrained YOLOv8 because they are absent from the COCO taxonomy. This highlights the necessity of full YOLOv8 fine-tuning on DhakaAI annotations as a crucial next step.

**Table 4.** YOLOv8n detection results on 12 DhakaAI test images across all four image sub-collections.

| SL. No | Source Set | Image ID      | Objects | Primary Classes         | Detected | Peak Conf. |
|--------|------------|---------------|---------|-------------------------|----------|------------|
| 1      | Asraf      | Asraf.85      | 11      | Car, Truck, Person      |          | 0.97       |
| 2      | Shykat     | Shykat_03_021 | 7       | Car, Motorcycle, Person |          | 0.94       |
| 3      | Sabiha     | sabiha(61)    | 7       | Car, Bus, Person        |          | 0.95       |
| 4      | Pias       | Pias(727)     | 6       | Car, Truck, Bus         |          | 0.96       |
| 5      | Sabiha     | sabiha(230)   | 6       | Car, Motorcycle, Person |          | 0.91       |
| 6      | Sabiha     | sabiha(144)   | 3       | Car, Bus                |          | 0.93       |
| 7      | Sabiha     | sabiha(219)   | 2       | Car                     |          | 0.88       |
| 8      | Sabiha     | sabiha(84)    | 2       | Car, Truck              |          | 0.85       |
| 9      | Shykat     | Shykat_5_(33) | 1       | Truck                   |          | 0.91       |
| 10     | Sabiha     | sabiha(259)   | 1       | Car                     |          | 0.82       |
| 11     | Pias       | Pias(864)     | 1       | Person                  |          | 0.86       |
| Total  |            |               | 47      |                         |          |            |

- The 0-detection result for Sabiha(174) reflects a genuinely sparse-traffic scene, confirming that the model does not produce excessive false positives.

Figures 3–6 display representative annotated detection outputs sampled across all four DhakaAI image sub-collections.

4.6 Synthesis: Combined Pipeline Assessment

Combining the MobileNetV2 classification and YOLOv8 detection results gives a clear picture of how a Dhaka traffic monitoring pipeline would work in real life.

**Table 5.** Assessment of MobileNetV2, YOLOv8, and the combined pipeline for Dhaka traffic monitoring.

| Component   | Strength                                                                      |                           |
|-------------|-------------------------------------------------------------------------------|---------------------------|
| MobileNetV2 | Fine-grained 21-class discrimination; lightweight (3.4M parameters)           | Cropped-patch input only; |
| YOLOv8      | Full-scene localization; real-time inference; high confidence on COCO classes | Cannot detect Dhaka       |
| Combined    | YOLOv8 proposes regions → MobileNetV2 classifies within-Dhaka classes         |                           |
|             |                                                                               | Requires YOLOv8           |

A perfect production setup would use YOLOv8 (fine-tuned on DhakaAI annotations) as the front-end detector for all 21 classes. MobileNetV2 could also be used as a lightweight re-classifier for three-wheeler and four-wheeler sub-groups that are hard to classify based on their shape.

## 5 Discussion

### 5.1 The Class Imbalance Challenge

The DhakaAI dataset exhibits a **power-law class distribution** typical of real-world traffic images. A few dominant classes (Car, Bus, Rickshaw, CNG) account for the majority of instances, while tail classes (Ambulance, Army Vehicle, Police Car, Wheelbarrow) are extremely rare. This distribution is not a sampling artifact; it accurately reflects the actual traffic composition in Dhaka. However, it poses a significant challenge for standard categorical cross-entropy training: during gradient accumulation, minority classes are overwhelmed by majority classes, leading to biased weight updates favoring the dominant classes. To address this, we need more than simple data augmentation. Class-weighted loss functions provide a first-order correction by scaling the loss contribution of each sample according to the inverse frequency of its class. Focal Loss [?], originally developed for dense object detection, down-weights easy (well-classified) examples and focuses training on hard (often minority) instances. This approach is particularly suitable for the DhakaAI long-tail scenario. Synthetic data generation using techniques such as Stable Diffusion or ControlNet can further balance the dataset by creating photorealistic images of minority classes (e.g., ambulances, police cars, and wheelbarrows), effectively equalizing their training set sizes relative to majority classes.

### 5.2 Sensitivity to Resolution and Input Scale

The uniform  $128 \times 128$  input resolution is computationally efficient but limits the distinguishability of fine-grained features, such as window structures, antennas, or lights on a police car roof. Increasing input resolution to  $224 \times 224$  or  $299 \times 299$ , standard for MobileNetV2 and Inception-V3, would raise computation costs by approximately  $3\text{--}5\times$ . Nevertheless, higher resolution is likely to improve intra-cluster discrimination. A principled ablation study across multiple resolutions is a direct avenue for future work.

### 5.3 Societal and Urban Policy Consequences

The motivation for this work and the DhakaAI Challenge extends beyond academic benchmarking. A production-ready Dhaka traffic detection system, deployable on edge devices, could enable:

- Automatic traffic signal timing optimization based on real-time traffic composition.
- Incident detection and emergency vehicle prioritization.
- Traffic density mapping to inform urban planning decisions.
- Enforcement of traffic regulations.
- Collection of traffic flow data to support infrastructure expansion, aiming to increase Dhaka’s 7% road coverage toward the 25% international urban planning standard.

The DhakaAI project positions such a system as a key component of a broader initiative to leverage AI for improved governance in Bangladesh and other rapidly urbanizing regions in South and Southeast Asia.

## 6 Conclusion

This paper presents the most thorough evaluation to date of deep learning-based multi-class traffic object detection on the **DhakaAI benchmark dataset**. This dataset was created specifically for traffic detection and captures the uniquely complex 21-class vehicle taxonomy of Dhaka, Bangladesh, one of the most congested cities in the world.

Our MobileNetV2 two-phase transfer learning pipeline achieved a classification accuracy of 66.34%, representing a 106.5% improvement over a naive baseline, on a 3,832-sample held-out test set. Training used over 19,000 bounding-box-cropped instances. Integration of **YOLOv8** for end-to-end full-image detection demonstrated strong localization confidence (0.82–0.97) on DhakaAI test images for globally common vehicle classes.

A systematic analysis revealed that the main barrier to higher accuracy is not the network’s capacity, but rather **extreme class imbalance**: categories with fewer than  $\sim 50$  instances achieve an F1 score of 0.00, while dominant classes achieve F1 scores of 0.75–0.79. Intra-cluster morphological confusion (four-wheelers, three-wheelers, large rectangular vehicles) indicates that input resolution and class-weighted training are the most impactful subsequent interventions.

## Future Directions

The five most important next steps are:

1. **Fine-Tuning YOLOv8 on DhakaAI Annotations:** Train YOLOv8 end-to-end on DhakaAI XML training data to detect regionally specific vehicles (CNG, Rickshaw, Human Hauler) absent from COCO.
2. **Synthetic Minority-Class Augmentation:** Use Stable Diffusion with Dhaka-context ControlNet conditioning to generate realistic Ambulance, Police Car, and Army Vehicle instances, balancing minority-class training data.
3. **Focal Loss and Class-Weighted Training:** Apply Focal Loss ( $\gamma = 2.0$ ) and per-class inverse-frequency weighting instead of standard categorical cross-entropy to prevent majority-class dominance in gradient updates.
4. **Higher Input Resolution Ablation:** Conduct a systematic study of  $128 \times 128$ ,  $224 \times 224$ , and  $299 \times 299$  input configurations to optimize accuracy versus compute cost for intra-cluster disambiguation.
5. **Edge Hardware Benchmarking:** Evaluate TFLite and ONNX quantization and deployment on Raspberry Pi 5 and NVIDIA Jetson Nano to measure real-world latency, throughput, and energy consumption for integrating Dhaka traffic cameras.

The DhakaAI research agenda is well-positioned to create production-grade, real-time traffic intelligence infrastructure to serve Dhaka’s 8 million daily commuters, while also providing a methodological template for other rapidly urbanizing cities in South and Southeast Asia.

## Acknowledgments

The authors thank **Shihavuddin, ASM, and Mohammad Rifat Ahmmad Rashid** for making the DhakaAI dataset publicly available, and **Harvard Dataverse** for hosting it. This project was conducted in the spirit of the DhakaAI Challenge’s mission to foster an AI research community in Bangladesh and Southeast Asia.

## Dataset

Shihavuddin, ASM; Mohammad Rifat Ahmmad Rashid, 2020, “DhakaAI,” <https://doi.org/10.7910/DVN/POREXF>, Harvard Dataverse, V1.

## References

1. Bangladesh Road Transport Authority (BRTA), 2023. *Annual Traffic Volume Statistics: Dhaka Metropolitan Area*. Ministry of Road Transport and Bridges, Government of Bangladesh.
2. Sustainable Urban Transport Project (SUTP), 2019. *Dhaka Transport Situation: Challenges and Policy Options*. GIZ Transport Policy Advisory Services.
3. World Bank, 2021. *Bangladesh: Dhaka Urban Transport Project — Situation Analysis*. World Bank Report No. 154982-BD.
4. Centre for Policy Dialogue (CPD), 2020. *Economic Cost of Traffic Congestion in Dhaka City*. Working Paper 127, CPD, Dhaka.
5. Wang, Y., et al., 2020. Deep Learning for Smart Connected Communities: A Survey. *International Journal of Computer Vision*, 128(2), 344–368.
6. Arnold, E., et al., 2019. A Survey on 3D Object Detection Methods for Autonomous Driving Applications. *IEEE Transactions on Intelligent Transportation Systems*, 20(10), 3782–3795.
7. Shihavuddin, ASM; Rashid, M.R.A., 2020. “DhakaAI [dataset]”, Harvard Dataverse, V1. <https://doi.org/10.7910/DVN/POREXF>
8. Lin, T.-Y., et al., 2014. Microsoft COCO: Common Objects in Context. *European Conference on Computer Vision (ECCV)*, Springer, 740–755.
9. Everingham, M., et al., 2010. The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision*, 88(2), 303–338.
10. Stauffer, C., Grimson, W.E.L., 1999. Adaptive Background Mixture Models for Real-Time Tracking. *Proceedings of CVPR 1999*, 2, 246–252.
11. Viola, P., Jones, M., 2001. Rapid Object Detection Using a Boosted Cascade of Simple Features. *Proceedings of CVPR 2001*, 1, I-511–I-518.
12. Dalal, N., Triggs, B., 2005. Histograms of Oriented Gradients for Human Detection. *Proceedings of CVPR 2005*, 1, 886–893.



13. Krizhevsky, A., Sutskever, I., Hinton, G.E., 2012. ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, 25.
14. Simonyan, K., Zisserman, A., 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR 2015*. arXiv:1409.1556.
15. Szegedy, C., et al., 2015. Going Deeper with Convolutions. *Proceedings of CVPR 2015*, 1–9.
16. He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep Residual Learning for Image Recognition. *Proceedings of CVPR 2016*, 770–778.
17. Girshick, R., Donahue, J., Darrell, T., Malik, J., 2014. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. *Proceedings of CVPR 2014*, 580–587.
18. Ren, S., He, K., Girshick, R., Sun, J., 2016. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *IEEE TPAMI*, 39(6), 1137–1149.
19. Lin, T.-Y., et al., 2017. Feature Pyramid Networks for Object Detection. *Proceedings of CVPR 2017*, 2117–2125.
20. Redmon, J., Divvala, S., Girshick, R., Farhadi, A., 2016. You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of CVPR 2016*, 779–788.
21. Redmon, J., Farhadi, A., 2018. YOLOv3: An Incremental Improvement. arXiv:1804.02767.
22. Jocher, G., et al., 2023. Ultralytics YOLOv8. Ultralytics. <https://github.com/ultralytics/ultralytics>
23. Howard, A.G., et al., 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861.
24. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C., 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. *Proceedings of CVPR 2018*, 4510–4520.
25. Howard, A., et al., 2019. Searching for MobileNetV3. *Proceedings of ICCV 2019*, 1314–1324.
26. Tan, M., Le, Q.V., 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*, 6105–6114.
27. Geiger, A., Lenz, P., Urtasun, R., 2012. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. *Proceedings of CVPR 2012*, 3354–3361.
28. Caesar, H., et al., 2020. nuScenes: A Multimodal Dataset for Autonomous Driving. *Proceedings of CVPR 2020*, 11621–11631.
29. Yu, F., et al., 2020. BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning. *Proceedings of CVPR 2020*, 2636–2645.
30. Ardianto, R., Chen, C.J., Hang, H.M., 2021. Traffic Monitoring System Using YOLOv4 and DeepSORT. *Procedia Computer Science*, 179, 74–81.
31. Mukherjee, S., et al., 2021. Indian Vehicle Classification Using Transfer Learning. *IEEE ICICT*, 1–6.
32. Khan, M.A., et al., 2020. Motorcycle Detection Using DeepLabV3. *IEEE Access*, 8, 140196–140207.
33. Zhang, Y., et al., 2022. Real-Time Traffic Object Detection on Edge Computing Devices Based on Lightweight Deep Learning. *IEEE Internet of Things Journal*, 9(4), 2824–2834.
34. Lin, T.-Y., et al., 2017. Focal Loss for Dense Object Detection. *Proceedings of ICCV 2017*, 2980–2988.